GraphR: Structure-Aware Multi-Context Embeddings and Retrieval for Binary Function Name Recovery

Ananta Dian Pradipta
New Jersey Institute of Technology
Newark, NJ, USA
adp232@njit.edu

Robert Blacha
New Jersey Institute of Technology
Newark, NJ, USA
rb97@njit.edu

Zhihao Lin
New Jersey Institute of Technology
Newark, NJ, USA
zl527@njit.edu

Haotian Zhang
New Jersey Institute of Technology
Newark, NJ, USA
haotian.zhang@njit.edu

ABSTRACT

Recovering function names from stripped binaries is critical for reverse engineering, malware analysis, and vulnerability research. Recent systems for this task increasingly rely on billion-parameter LLM backbones. On our evaluation setup, we show a small, structure-aware encoder can match or outperform them. We present GraphR, a graph neural network-based system that learns function representations by fusing multiple context sources (external library calls, callee signatures, and caller signatures) through a cascaded gated fusion mechanism with conditional bypass. Our ablation study shows that external-call information alone improves in-distribution performance but reduces cross-project generalization by 4.3 points. Adding callee and caller context offsets this effect and improves disambiguation. We further show that self-supervised pre-training (MLM plus contrastive learning on O0/O2 pairs of the same function) paired with sufficient encoder capacity (we use 32M parameters; smaller variants benefit less from pre-training) delivers the largest single ablation gain: **+0.13 cross-project F1**. In our experiments, nearest-neighbor retrieval over encoder embeddings outperformed the autoregressive decoder on unseen packages, improving exact match by 4.7 points without additional training. This result suggests that, for our model and dataset, embedding quality was more important than the decoding strategy. Evaluated on 300,013 functions from 77 packages, GraphR achieves **0.770 F1** on a held-out test set (13,559 functions) and **0.718 F1** across 4 completely unseen packages (10,467 functions), outperforming the released SYMGEN checkpoint (0.45), an end-to-end SYMGEN+LoRA reproduction fine-tuned on our full 300K training data (0.66), BLens (0.46 reported), and a StarCoder-3B zero-shot LLM baseline (0.65). On a matched 8,062-function subset, GraphR leads SYMGEN+LoRA by **+0.10 F1** and **+26 pp EM**. Our 32M-parameter model is over 1,000× smaller than SYMGEN's CodeLlama-34B backbone.

KEYWORDS

binary analysis, function name recovery, graph neural networks, gated fusion, retrieval-augmented inference

1 INTRODUCTION

Software reverse engineering is a fundamental activity in cybersecurity, requiring analysts to understand the behavior of compiled binary programs without access to source code. Modern compilers strip all symbolic information (function names, variable names, type annotations) from release binaries, leaving analysts with thousands of unnamed functions (e.g., `sub_4a30`, `FUN_00105c10`) that must be manually identified. Recovering meaningful function names dramatically accelerates program comprehension: knowing that `sub_4a30` is `xmalloc` immediately conveys its purpose.

The practical importance of automated function name recovery extends across multiple domains of cybersecurity. In malware analysis, meaningful function names reduce triage time from hours to minutes, allowing analysts to quickly identify cryptographic routines, network communication handlers, and persistence mechanisms. In vulnerability research, understanding stripped firmware is essential for discovering exploitable bugs in IoT devices and embedded systems, where vendors rarely ship debug symbols. In competitive reverse engineering (CTFs) and forensic investigations, even partial name recovery provides critical footholds for understanding program logic. Commercial tools such as IDA Pro and Ghidra provide manual annotation workflows and limited pattern-matching (FLIRT signatures), but cannot automatically recover names for arbitrary functions. A comprehensive survey of binary code similarity techniques is given by Haq and Caballero [64].

Prior work on automated function name recovery has explored statistical approaches [1], extreme multi-label classification [35], graph neural networks [2], language model pre-training [3], and large language model fine-tuning [5]. However, two fundamental challenges remain underexplored: (1) how to effectively combine multiple sources of contextual information (code structure, library calls, and inter-procedural relationships) without degrading generalization, and (2) whether autoregressive name generation is the right output paradigm, given that function names are drawn from a relatively small vocabulary of real names.

We address these challenges with GraphR, a graph neural network system that lifts stripped binaries to BAP intermediate representation, encodes each function's control flow graph with a GAT-based encoder, and fuses multiple sources of inter-procedural context through a cascaded gated mechanism with conditional bypass. Self-supervised pretraining (masked language modeling and contrastive learning) with partial embedding transfer further strengthens the encoder representations. At inference time, we demonstrate that simple k-nearest-neighbor retrieval over the learned embeddings outperforms autoregressive beam search decoding, improving

cross-project exact match by 4.7 percentage points with zero retraining.

Evaluated on 300,013 functions from 77 packages, GraphR achieves 0.770 F1 on a held-out test set (13,559 functions) and **0.718 F1** across 4 completely unseen packages (10,467 functions), outperforming the released SYMGEN [5] checkpoint (0.45), an end-to-end SYMGEN+LoRA reproduction fine-tuned on our full 300K training set (0.66), BLens [4] (0.46 reported), and a StarCoder-3B zero-shot LLM baseline (0.65), while using only 32M parameters, over 1,000× smaller than SYMGEN's CodeLlama-34B backbone.

In summary, we make the following contributions:

- We propose GraphR, a GNN-based system for binary function name recovery that uses staged gated fusion to combine multiple context sources. This 3-stage architecture sequentially fuses external library calls, callee signatures, and caller signatures with conditional bypass, where each stage is skipped when the corresponding context is absent. Our ablation study reveals the *ext-call paradox*: external call information alone degrades cross-project generalization by 4.3 percentage points, while adding interprocedural callee/caller context resolves this and improves cross-project EM by 12.8 percentage points.
- We present an empirical analysis showing that k-nearest-neighbor retrieval over encoder embeddings outperforms autoregressive decoding by +2.0pp test EM and +4.7pp cross-project EM with zero retraining. This finding, consistent with kNN-LM [16], demonstrates that for our encoder architecture, representation quality is the primary bottleneck rather than the decoding strategy.
- We demonstrate that a compact, structure-aware encoder can match or outperform billion-parameter LLM-based baselines on this task. GraphR uses 32M parameters (over 1,000× fewer than SYMGEN's CodeLlama-34B backbone), trains in under 2.5 hours on a single A100-80GB GPU, and serves predictions in single-digit milliseconds per function, an order of magnitude faster than autoregressive decoding from a 34B LLM. This makes GraphR practical for interactive reverse-engineering workflows.
- We show that self-supervised pre-training (masked language modeling on instruction tokens, plus contrastive learning on O0/O2 pairs of the same function) paired with sufficient encoder capacity (we use 32M parameters; smaller variants benefit less from pre-training) delivers the largest single ablation gain: **+0.13 cross-project F1** from our 8M-parameter ablation baseline. Partial embedding transfer (84.4% of token embeddings matched by name) lets us reuse pre-training across vocabulary changes between pre-training and fine-tuning runs, reducing the practical cost of re-training when the dataset evolves.
- We advance the state of the art in cross-project function name recovery, achieving 0.770 F1 on a held-out test set (13,559 functions) and **0.718 F1** across 4 completely unseen packages (10,467 functions). This exceeds the released SYMGEN [5] checkpoint (0.45), an end-to-end SYMGEN+LoRA reproduction on our full 300K training data (0.66), BLens [4] (0.46 reported), and a StarCoder-3B zero-shot LLM baseline (0.65). On a matched 8,062-function subset we lead SYMGEN+LoRA by +0.10 F1 / +26 pp EM while using 1,000× fewer parameters. Our code and dataset are publicly available at https://github.com/ananta-pradipta/stripped-binary-name-recovery.

2 BACKGROUND AND MOTIVATION

2.1 Problem Definition

We formalize binary function name recovery as follows. Given a stripped binary B containing n functions $\{f_1, f_2, \ldots, f_n\}$, where each function f_i is represented by its control flow graph $G_i = (V_i, E_i)$ with basic blocks as nodes and control flow edges, the goal is to predict a human-readable name $\hat{n}_i$ for each function that matches the developer-assigned name n_i from the original source code.

Each function name is decomposed into a sequence of sub-tokens using a vocabulary $\mathcal{V}$ of 2,642 tokens: $n_i = (t_1, t_2, \ldots, t_L)$ where $t_j \in \mathcal{V}$ and $L \leq 20$. For example, `quotearg_buffer_restyled` becomes the sub-token sequence [`quote`, `arg`, `_`, `buffer`, `_`, `re`, `styled`]. The prediction task is thus a sequence generation problem conditioned on the function's binary representation.

In addition to the CFG, each function may have three forms of contextual information: (1) external library calls $\mathcal{E}_i$, the set of PLT/IAT function names called by f_i (e.g., `malloc`, `fprintf`); (2) callee signatures C_i^{ee}, instruction-type token sequences of internal functions called by f_i; and (3) caller signatures C_i^{er}, instruction-type token sequences of functions that call f_i. These contextual signals are optional: approximately 70% of functions have no external calls, and leaf functions have no callees.

2.1.1 Scope and Assumptions. We scope GraphR to the following setting:

- **Architecture**: x86-64 ELF binaries compiled for Linux.
- **Optimization levels**: O0, O1, O2, and O3, plus a default level for binaries that override `-O` flags.
- **Compilation**: GCC compiler. We do not evaluate Clang or MSVC, though the instruction-type tokenization is compiler-agnostic at the IR level.
- **Training corpus**: We assume access to a corpus of labeled binaries where ground-truth function names are known (obtained from debug builds). This is a standard assumption in the literature [1–5].
- **No obfuscation**: We assume binaries are not deliberately obfuscated. Obfuscation techniques (control flow flattening, opaque predicates, symbol stripping with anti-disassembly) would require additional preprocessing.

2.1.2 Evaluation Scenario. We evaluate in two settings: (1) a held-out test set of 13,559 functions from binaries of packages seen during training (cross-binary evaluation), and (2) a cross-project set of 10,467 functions from 4 packages (tengine, angie, nginx118, recutils) completely excluded from training across all four GCC optimization levels (O0/O1/O2/O3). The cross-project setting is stricter than the cross-binary evaluation used by some prior work [2, 3], where different binaries of the *same* package may appear in both training and test. Cross-project evaluation tests whether the model generalizes to entirely new codebases with potentially novel

naming conventions, and is the evaluation methodology advocated by SYMGEN [5].

2.2 Challenges

Binary function name recovery presents several fundamental challenges that motivate our design decisions:

Token collision. After instruction-type tokenization, many structurally different functions share similar token distributions. Two unrelated functions may both consist primarily of `MEM_READ_64`, `ARITH_ADD`, and `STACK_STORE_64` tokens, making them difficult to distinguish from instruction tokens alone. This motivates our multi-context fusion: inter-procedural context (what a function calls and who calls it) provides disambiguating signal that code structure alone cannot.

Mode collapse. When contextual features are naively incorporated, the fusion mechanism must produce meaningful output even when no context exists (e.g., a function with no external calls receives a zero-vector context input). In practice, this causes the gate to learn a default output, and 47% of all predictions collapse to a single name (`xmalloc`, the most frequent function). Our conditional bypass mechanism addresses this by skipping fusion stages entirely when context is absent.

Cross-project generalization gap. Models that perform well on held-out test sets from seen packages often degrade dramatically on entirely unseen packages. BLens [4] drops from 0.79 to 0.46 F1 (42% degradation), and our own ablation reveals that external call information (which helps on seen binaries) actually *hurts* cross-project generalization by 4.3 percentage points (the ext-call paradox). This motivates careful fusion design with conditional bypass and inter-procedural disambiguation.

O0/O2 optimization gap. Binaries compiled without optimization (O0) produce many small wrapper functions with similar boilerplate stack frame setup/teardown, where two unrelated functions may have 70% token overlap from stack operations alone. O2-compiled code eliminates this boilerplate through register allocation and inlining, leaving more distinctive patterns. This gap (41.5% vs. 69.8% EM on our test set) motivates our contrastive pretraining objective that learns optimization-invariant representations.

2.3 External Calls

External calls exist in a program in order to allow for programs to use common library functions without needing to duplicate those functions across multiple binaries and to allow for updating libraries to be handled separately from updating individual programs. For the linker to be able to link a binaries external calls to the function in a loaded library (.dll in windows and .so in linux), it must retain the information of the function's name to link it to the appropriate function in a loaded library, even when that binary is stripped, optimized or even obfuscated. While other state of the art function name predictors handle all call instructions as the same token, GraphR is able to extract two vital pieces of information from separately collecting and analyzing the external calls within a function:

(1) The called external functions within an unnamed function, providing context on what the function is attempting to accomplish.
(2) The calling context of a function by reference to its callers and callees external function calls, allowing for functions without external function calls to gain the benefit from (1).

2.4 Related Work and Our Motivations

2.4.1 Binary Function Name Recovery. Patrick-Evans et al. [35] framed function naming as extreme multi-label classification (XFL), treating each sub-token as an independent label, but this closed-vocabulary formulation cannot produce names outside the label set. He et al. [1] introduced DeBin, which uses BAP intermediate representation with probabilistic graphical models (ExtraTrees + CRF) to predict debug information including function names. While pioneering, DeBin's statistical approach cannot capture the complex patterns that deep learning exploits. We use DeBin as the non-neural baseline in our ablation (Table 2).

David et al. [2] proposed NERO, using pointer-aware slicing and GNNs on call graphs to predict function names, achieving F1=0.455, building on source-level method naming work such as code2seq [54]. NERO's key insight is that call-site context matters for naming. However, NERO only works for functions *with* external calls, a fundamental limitation since 70% of functions in typical binaries have none. Our conditional bypass mechanism (Section 4.4) addresses this directly: functions without calls use only code features, while functions with calls benefit from multi-context fusion.

Jin et al. [3] presented SymLM, which pre-trains on micro-execution traces and fuses caller/callee context for function name prediction (F1=0.585). SymLM's "fusing encoder" concatenates context embeddings, whereas our gated fusion learns a per-function balance between code and context features with conditional bypass. More critically, SymLM requires dynamic analysis infrastructure (executing binaries in an emulator), whereas GraphR operates purely on static analysis. Our ablation reveals the ext-call paradox (Section 5.2), a phenomenon SymLM does not analyze, showing that context fusion must be carefully designed to avoid degrading generalization.

Gao et al. [49] proposed NFRE, a lightweight framework for function name reassignment that demonstrated scalability to large stripped binaries. Al-Kaswan et al. [4] introduced BLens, combining ensemble embeddings (including PalmTree [39] instruction embeddings) with contrastive captioning. While achieving 0.79 F1 per-binary, cross-project F1 drops to 0.46, a 42% degradation that highlights the generalization challenge we address. Xu et al. [5] fine-tuned CodeLlama [28] with LoRA [29] for function naming, importantly demonstrating that prior results were inflated 1.2–2.9× by duplicate functions (cross-project F1=0.38 after proper deduplication). We follow SYMGEN's deduplication methodology in our evaluation. Su et al. [36] (ProRec) showed that source-code foundation models are transferable knowledge bases for binary analysis, further motivating the LLM-based baseline comparison.

Jiang et al. [6] applied domain-adapted LLMs to function name inference, representing the trend toward large-scale language model approaches. GenNm [7] fine-tuned CodeGemma with SymPO preference optimization for variable name recovery, a related task demonstrating that preference-aligned generation improves binary symbol prediction. Related work on decompiled identifier naming includes DIRE [45] and DIRTY [46], which predict variable names

and types from decompiler output, while VarBERT [47] applies transfer learning to variable name prediction and ReSym [48] harnesses LLMs for variable and data structure symbol recovery. Unlike recent LLM-based approaches, we focus on a compact encoder and compare retrieval-based inference against autoregressive decoding, consistent with Khandelwal et al.'s finding that retrieval can outperform generation when representations are sufficiently good [16, 17].

2.4.2 Binary Code Representation Learning. Learning effective representations of binary code underpins all downstream binary analysis tasks, including function boundary detection [44], disassembly [52], and neural decompilation [62]. Xu et al. [8] pioneered GNN-based binary analysis with Gemini, using graph embedding networks for cross-platform binary similarity detection. We adopt their insight that CFG structure is informative, extending it with GAT attention and multi-context fusion. Ding et al. [9] proposed Asm2Vec, learning function embeddings via PV-DM [33] random walks over assembly instructions. Unlike Asm2Vec's raw instruction tokens, our V3 tokenization abstracts instructions into 1,510 semantic types, reducing vocabulary by 95% while preserving discriminative information.

Wang et al. [10] introduced jTrans, a jump-aware Transformer that incorporates control flow structure into positional encoding for binary similarity. Pei et al. [11] developed Trex, learning execution semantics from micro-traces. Massarelli et al. [53] proposed SAFE, using self-attentive function embeddings for binary similarity. Wang et al. [40] introduced CLAP, which learns transferable binary code representations through natural language supervision via contrastive learning. Xiong et al. [50] proposed HexT5, a unified pre-training framework for multiple stripped binary code inference tasks, and Jiang et al. [51] introduced Nova, a generative assembly language model with hierarchical attention. These works demonstrate the value of pre-training on binary code. We adopt this principle in our MLM + contrastive pre-training stage (Section 4.5), but operate on static BAP-IR rather than execution traces.

Zhu et al. [12] proposed CALLEE for recovering call graphs via contrastive learning with transfer learning, showing +28% F1 from pre-training on related tasks. Two of their findings directly support our design: (1) transfer learning from pre-training provides substantial gains, validating our encoder pre-training approach; (2) "loose symbolization" (semantic type abstraction) outperforms strict tokenization, supporting our instruction-type tokenization.

2.4.3 GNNs for Binary Analysis. Graph neural networks [57–59] are a natural fit for binary analysis, where control flow graphs (CFGs) provide structural information. Zhu et al. [13] applied GNNs to type inference on stripped binaries (TYGR), demonstrating that graph structure captures semantic properties beyond sequential instruction patterns. He et al. [14] proposed semantics-oriented graph representations for binary code similarity, arguing that "code is not natural language" and that graph-based approaches better capture program semantics than sequential models.

Yu et al. [38] proposed CFG2VEC, a hierarchical GNN for cross-architectural reverse engineering that learns function representations from CFGs. Our work extends GNN-based binary analysis in two ways: (1) we use graph attention networks (GAT) with learned attention [61] over CFG edges, allowing the model to weight structurally important blocks; (2) we introduce a multi-context fusion mechanism that incorporates inter-procedural information (callee/caller signatures and library calls) not present in the CFG alone.

2.4.4 Retrieval-Augmented Generation. Our finding that k-NN retrieval outperforms autoregressive decoding connects to a growing body of work on retrieval-augmented methods in NLP. Khandelwal et al. [16] demonstrated that augmenting a neural language model with a nearest-neighbor search over cached hidden states (kNN-LM) significantly improves perplexity without any additional training. They extended this to machine translation (kNN-MT) [17], showing that retrieval-augmented translation outperforms the base model, particularly on domain-specific data. Borgeaud et al. [18] proposed RETRO, which retrieves relevant text chunks from a large database during generation, achieving performance comparable to models 25× larger.

These works share a common insight with our findings: when the output space is drawn from a fixed set of real examples (function names in training, sentences in a corpus), retrieval over learned representations can outperform unconstrained generation. However, our setting differs in an important way: in kNN-LM/kNN-MT, retrieval *augments* generation at the token level, whereas in our system, retrieval *replaces* generation entirely: we retrieve a complete function name rather than interpolating token-level distributions. This is because function names are atomic labels (each name is a single concept), not compositional sequences where token-level retrieval would help.

3 OVERVIEW

GraphR follows a 3-stage encoder-decoder [32] pipeline, illustrated in Figure 1. The pipeline transforms a stripped binary into predicted function names through three phases:

Stage 1: Block Encoding. Each function's basic blocks are tokenized into instruction-type tokens (Section 4.1.1), which are processed by a Transformer encoder to produce per-block embeddings $\mathbf{b}_i \in \mathbb{R}^{512}$.

Stage 2: Graph Encoding and Multi-Context Fusion. A Graph Attention Network (GAT) aggregates block embeddings over the control flow graph to produce a function-level embedding $\mathbf{f} \in \mathbb{R}^{1024}$. The cascaded gated fusion mechanism then sequentially incorporates external library calls, callee signatures, and caller signatures through learned gates with conditional bypass, producing the final fused embedding $\mathbf{z} \in \mathbb{R}^{1024}$.

Stage 3: Name Prediction. The fused embedding $\mathbf{z}$ feeds either a GRU decoder with beam search or k-NN retrieval over the training set embeddings, producing a predicted function name as a sequence of sub-tokens.

Prior to supervised training, the block encoder and graph encoder are jointly pre-trained with two self-supervised objectives: masked language modeling (predicting randomly masked instruction tokens) and contrastive learning (pulling together embeddings of the same function compiled at O0 and O2). This pre-training, followed by partial embedding transfer (84.4% of token embeddings matched by name), provides the encoder with a strong initialization that improves cross-project F1 by 0.13.

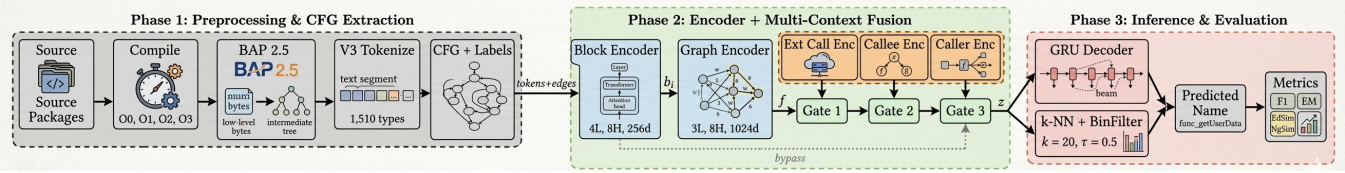

Figure 1: End-to-end GraphR pipeline. Phase 1 compiles source packages, lifts to BAP-IR, tokenizes instructions into 1,510 semantic types, and extracts CFGs with ground-truth labels. Phase 2 encodes basic blocks (Transformer), aggregates over the CFG (GAT), and fuses library call context, callee signatures, and caller signatures through cascaded gated fusion with conditional bypass. Phase 3 produces function names via either GRU beam search or k-NN retrieval, evaluated with sub-token F1, exact match, edit similarity, and n-gram similarity.

Figure 2 shows the detailed model architecture. The key design decision is the cascaded gated fusion with conditional bypass (Figure 3): each fusion stage is skipped entirely when the corresponding context is absent, preventing mode collapse.

4 DETAILED DESIGN

4.1 Preprocessing

We compile source packages at optimization levels O0, O1, O2, and O3, producing both debug and stripped binaries. We use the BAP 2.5.0 [15] binary analysis platform (alternatives include angr [43]) to lift stripped binaries to BAP Intermediate Representation (BAP-IR), which provides a normalized, architecture-independent view of the binary code.

4.1.1 Instruction-Type Tokenization. Raw BAP-IR contains over 32,000 unique tokens, many of which are register names, immediate values, or memory addresses that vary across binaries. We classify each BAP-IR instruction into one of approximately 1,510 semantic types:

- **Calls**: `CALL_malloc`, `CALL_printf`, `CALL_INTERNAL`
- **Memory**: `MEM_READ_32`, `MEM_WRITE_ARG_64`, `MEM_READ_GLOBAL_32`
- **Flags**: `FLAG_CF`, `COND_BRANCH_ZF`
- **Constants**: `ASSIGN_ZERO`, `ASSIGN_POW2`, `ASSIGN_ADDR` (immediate value bucketing)
- **Control**: `BRANCH`, `RETURN`, `COMPARE`

This tokenization reduces vocabulary by 95% while preserving semantic distinctions. Our ablation shows a +1,750% F1 gain over raw BAP-IR tokens. The tokenization is deterministic and compiler-agnostic: it depends only on BAP-IR semantics, not on specific register assignments or memory layouts.

4.1.2 Dataset Construction. Our dataset is constructed through a multi-step pipeline. First, we compile each source package twice: a *debug* build (with `-g` flag) and a *stripped* build (with `strip -strip-all`). The debug build provides ground-truth function names via the `nm` utility, which extracts symbol table entries mapping addresses to names. The stripped build is the input to our model and contains no symbolic information.

We lift each stripped binary using BAP 2.5.0 [15], which produces BAP-IR files containing per-function control flow graphs. BAP names functions using their entry addresses (e.g., `sub_4a30`). We match BAP function addresses to debug symbols via exact address matching, discarding functions that cannot be matched (typically compiler-inserted functions like `_start` or `__libc_csu_init`).

Data is split at the *binary* level, not the function level, to prevent data leakage. A fixed split assignment file ensures reproducibility. Four entire packages (tengine, angie, nginx118, recutils, across all their binaries and optimization levels) are held out as the cross-project set and never appear during training. This split design means that the same function (e.g., `xmalloc` from gnulib) may appear in both training and test binaries. This is intentional, as it reflects the real-world scenario where shared library functions recur across packages.

4.1.3 Votes Name Tokenizer. Function names must be decomposed into sub-tokens for sequence prediction. Standard BPE [34] tokenization produces poor results because function names follow naming conventions (snake_case, camelCase) that differ from natural language. We developed Votes, a 3-model voting tokenizer (independently contemporaneous with Epitome's multi-model boundary voting [37]) that achieves 95% lower out-of-vocabulary rate compared to BPE.

Votes works as follows: three independent segmentation models (a character bigram model, a frequency-based model, and a rule-based model splitting on underscores, camelCase boundaries, and digit transitions) each propose sub-token boundaries for a given function name. A boundary is accepted if at least 2 of 3 models agree. This produces a vocabulary of 2,642 sub-tokens that covers 99.7% of all function name tokens in the dataset, compared to 94.8% for BPE with a comparable vocabulary size. The reduced OOV rate directly improves exact match: a name cannot be correctly predicted if any of its sub-tokens are out of vocabulary.

For example, `quotearg_buffer_restyled` is tokenized as [`quote`, `arg`, `_`, `buffer`, `_`, `re`, `styled`] by Votes, whereas BPE might produce [`quo`, `tear`, `g_buf`, `fer_rest`, `yled`], losing semantic boundaries.

4.1.4 ENDBR64 Wrapper Resolution. In O0-compiled binaries, BAP lifts `endbr64; jmp target` sequences as separate 2-block functions. These are Intel CET (Control-flow Enforcement Technology) stubs, not real functions. We found that 96.4% of O0 functions were such wrappers, each containing only a single `CALL_INTERNAL` token. Our preprocessing detects these wrappers and replaces their graphs with the target function's graph. On the 87K ablation subset, this raised O0 exact match from 1.6% to 41.5%.

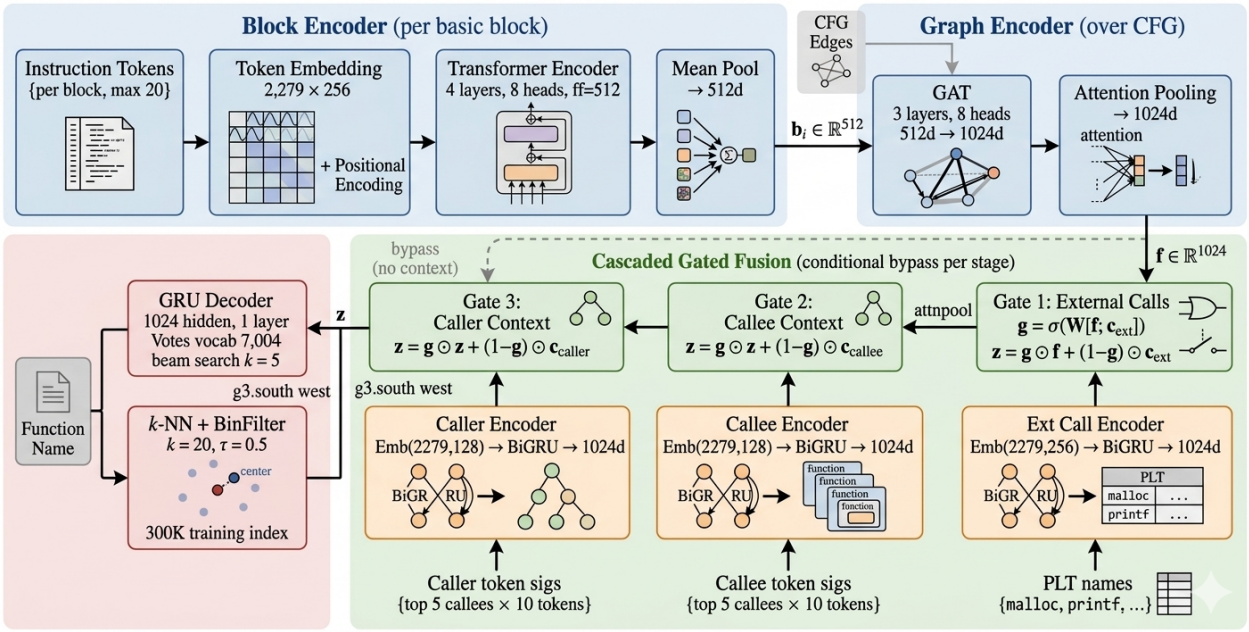

Figure 2: Detailed GraphR model architecture. The block encoder processes instruction-type tokens per basic block via a Transformer. The graph encoder aggregates block embeddings over the CFG using GAT with attention pooling. The cascaded gated fusion sequentially incorporates external library calls, callee signatures, and caller signatures, where each stage is bypassed when the corresponding context is absent. The fused embedding z feeds either a GRU decoder (beam search) or k-NN retrieval.

4.2 Block Encoder

Each basic block b contains a sequence of up to 20 instruction-type tokens $(x_1, x_2, \ldots, x_T)$ where $T \leq 20$. Blocks longer than 20 tokens are truncated; in practice, 95% of blocks have fewer than 20 tokens. We embed each token via a learned embedding table $\mathbf{E} \in \mathbb{R}^{2279 \times 256}$ and add sinusoidal positional encodings to preserve token order within the block:

$$\mathbf{h}_t^{(0)} = \mathbf{E}[x_t] + \text{PE}(t) \tag{1}$$

A Transformer encoder [21] with 4 layers and 8 attention heads (d_{model}=256, d_{ff}=512) processes the token sequence with multi-head self-attention:

$$\mathbf{H}^{(\ell)} = \text{TransformerLayer}^{(\ell)}(\mathbf{H}^{(\ell-1)}) \tag{2}$$

We apply mean pooling over the final-layer token representations to produce the block embedding:

$$\mathbf{b}_i = \text{Linear}\left(\frac{1}{T}\sum_{t=1}^{T} \mathbf{h}_t^{(L)}\right) \in \mathbb{R}^{512} \tag{3}$$

We chose mean pooling over alternative aggregation strategies after extensive experimentation. Token-level attention pooling (a weighted sum using learned attention scores) achieved only 0.34 F1, significantly worse than mean pooling's 0.58 F1. We hypothesize that attention pooling overfits to specific token positions, while mean pooling provides a more robust representation by treating all instruction types as equally important at the block level. This is consistent with the finding that instruction-type tokens are already semantically abstracted; further attention-based selection within a block provides diminishing returns.

A dropout [26] rate of 0.15 is applied to the Transformer layers during training for regularization. Each function may contain up to 30 basic blocks; functions with fewer blocks are padded, and a block-level attention mask ensures padding blocks do not influence the graph encoder.

4.3 Graph Encoder

Given a function's control flow graph $G = (V, E)$ where nodes are basic blocks with embeddings $\{\mathbf{b}_1, \ldots, \mathbf{b}_{|V|}\}$, we apply a 3-layer Graph Attention Network (GAT) [20] with 8 attention heads per layer. Each GAT layer computes attention-weighted message passing along CFG edges:

$$\alpha_{ij}^{(k)} = \frac{\exp\left(\text{LReLU}\left(\mathbf{a}^{(k)\top}[\mathbf{W}^{(k)}\mathbf{b}_i \| \mathbf{W}^{(k)}\mathbf{b}_j]\right)\right)}{\sum_{j' \in \mathcal{N}(i)} \exp\left(\text{LReLU}\left(\mathbf{a}^{(k)\top}[\mathbf{W}^{(k)}\mathbf{b}_i \| \mathbf{W}^{(k)}\mathbf{b}_{j'}]\right)\right)} \tag{4}$$

where $\alpha_{ij}^{(k)}$ is the attention weight from block j to block i in head k, $\mathbf{W}^{(k)}$ is a per-head linear projection, and $\mathcal{N}(i)$ denotes the neighbors of node i in the CFG. The multi-head outputs are concatenated (layers 1–2) or averaged (final layer), with residual connections [30] and layer normalization [31] between layers.

The GAT captures structural patterns such as loop back-edges (blocks that loop to themselves or to earlier blocks), conditional

branches (nodes with two outgoing edges), and sequential fall-through (single-successor blocks). These structural patterns are informative for naming: for instance, functions with deep loop nesting often implement search or sorting algorithms.

An attention-based readout pools over all block embeddings to produce the function-level embedding:

$$\mathbf{f} = \sum_{i=1}^{|V|} \beta_i \cdot \mathbf{b}_i^{(L)}, \quad \beta_i = \frac{\exp(\mathbf{w}^\top \mathbf{b}_i^{(L)})}{\sum_j \exp(\mathbf{w}^\top \mathbf{b}_j^{(L)})} \tag{5}$$

where $\mathbf{w}$ is a learned attention vector. This allows the model to focus on structurally important blocks (e.g., loop headers, error-handling blocks) rather than weighting all blocks equally. The output is projected to $\mathbf{f} \in \mathbb{R}^{1024}$.

4.4 Cascaded Gated Fusion

The core architectural contribution is a 3-stage cascaded fusion mechanism that sequentially incorporates contextual information (Figure 3):

Stage 1: External Calls. An external call encoder processes the sequence of library function names (e.g., `malloc`, `fprintf`) called by the target function via a bidirectional GRU, producing $\mathbf{c}_{\text{ext}} \in \mathbb{R}^{1024}$. A sigmoid gate determines the balance:

$$\mathbf{g} = \sigma(\mathbf{W}_g[\mathbf{f}; \mathbf{c}_{\text{ext}}] + \mathbf{b}_g) \tag{6}$$

$$\mathbf{z} = \begin{cases} \mathbf{g} \odot \mathbf{f} + (1 - \mathbf{g}) \odot \mathbf{c}_{\text{ext}} & \text{if has ext calls} \\ \mathbf{f} & \text{otherwise (bypass)} \end{cases} \tag{7}$$

Stage 2: Callee Context. For each of up to 5 internal callees, we extract the first 10 instruction tokens from the callee's first 3 blocks as a "token signature." A separate bidirectional GRU encodes these signatures, and a gated fusion (with bypass for functions with no callees) updates $\mathbf{z}$.

Stage 3: Caller Context. Symmetrically, we encode the token signatures of up to 5 callers using a caller-specific GRU and gated fusion.

Conditional bypass is critical. Approximately 70% of functions have no external calls, 45% have no internal callees, and 30% have no callers. Without bypass, the fusion gate must learn to produce a meaningful output even when the context input is a zero vector (no context). In practice, the gate learns a default output for these functions, causing 47% of all predictions to collapse to a single name (`xmalloc`, the most frequent function in the training set). This mode collapse is catastrophic for accuracy.

With bypass, the code representation $\mathbf{f}$ passes through unchanged when no context exists, preserving the discriminative information learned by the block and graph encoders. The conditional bypass can be viewed as a form of residual learning: each fusion stage can only *improve* the representation (by incorporating relevant context), never degrade it (by forcing a default when no context exists).

The cascade order (external calls → callees → callers) was chosen based on information specificity. External library calls provide the most specific signal (e.g., a function calling `socket` and `bind` is likely network-related). Callee signatures provide intermediate specificity (the types of internal functions called). Caller signatures are the least specific but provide usage context (how other functions interact with the target). Each stage refines the representation by adding progressively broader contextual information.

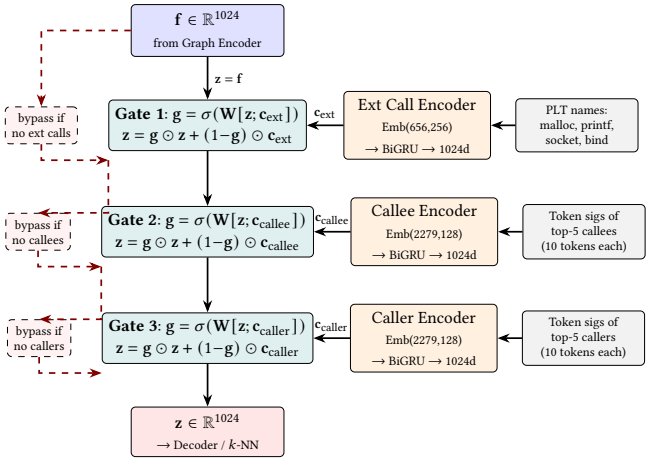

Figure 3: Cascaded gated fusion with conditional bypass. Each stage fuses a different context source via a learned sigmoid gate. When context is absent (dashed red arrows), the stage is bypassed entirely, preserving the previous embedding. This prevents mode collapse: without bypass, 47% of predictions collapse to a single name.

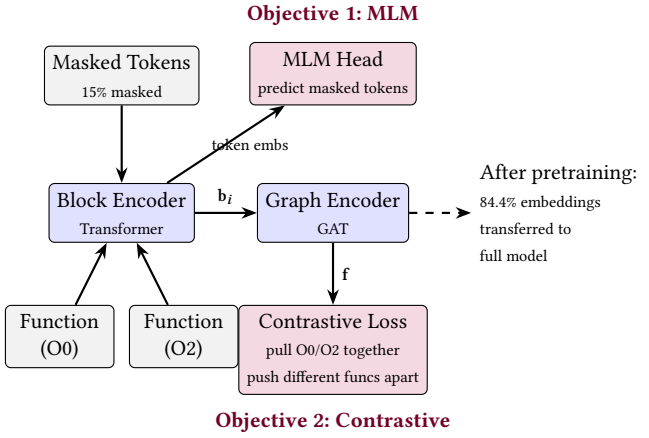

Figure 4: Self-supervised pretraining. Two objectives: (1) MLM predicts masked instruction tokens from block encoder token embeddings (before pooling), training the block encoder; (2) contrastive learning (NT-Xent) pulls together function-level embeddings (after graph encoder) of the same function at O0 and O2, training both encoders. After pretraining, 84.4% of token embeddings are transferred to the full model by matching token names.

4.5 Self-Supervised Pre-training

We pre-train the block encoder and graph encoder jointly with two self-supervised objectives before the full model is trained (Figure 4).

Masked Language Model (MLM). Following the BERT [22] pre-training paradigm adapted for binary code, we randomly mask 15% of instruction tokens in each basic block and train the model to predict the original tokens from the surrounding context. The MLM objective encourages the block encoder to learn contextual representations of instruction types.

Contrastive Learning. The same function compiled at O0 and O2 should produce similar embeddings, while different functions should produce dissimilar embeddings. We use the NT-Xent (Normalized Temperature-scaled Cross-Entropy) loss [23] with a temperature of τ=0.07. This objective is particularly valuable because it teaches the encoder that the same function can look different at different optimization levels, encouraging optimization-invariant representations.

The pre-training encoder has 12M parameters (smaller than the full 32M model, as the context encoders and decoder are not pre-trained). After 10 epochs of pre-training on the full training set, we transfer weights to the full model. We perform *partial embedding transfer*: 84.4% of token embeddings (1,923 out of 2,279 rows) are copied by matching token names between the pre-training and fine-tuning vocabularies; the remaining 356 tokens are randomly initialized. All Transformer and GAT weights are transferred directly.

4.6 Decoder and k-NN Inference

4.6.1 GRU Decoder. A single-layer GRU [24] decoder with 1024-dimensional hidden state generates function names as sub-token sequences. Training uses scheduled sampling (teacher forcing ratio 1.0⟶0.3) and label smoothing [25] (0.1). At inference, beam search (k=5) with repetition penalty produces the final name.

4.6.2 k-NN Retrieval. Inspired by kNN-LM [16] and kNN-MT [17], which demonstrated that nearest-neighbor retrieval over hidden-state datastores can outperform neural language models and machine translation systems, we explore k-NN retrieval as an alternative output head. At inference, we build an index of all 300,013 training function embeddings. For each query, we compute its encoder embedding $\mathbf{z}$, retrieve the top-20 most similar training functions by cosine similarity, drop candidates whose parent training binary has Jaccard external-call-fingerprint overlap with the query binary below τ=0.5 (the BinFilter), and return the top-1 survivor's name. BinFilter lifts cross-project F1 from 0.698 (unfiltered k-NN) to 0.718 with zero retraining.

4.7 Training Details

We train on 300,013 functions from 77 packages spanning 851 binaries at four GCC optimization levels (O0: 47%, O1: 16%, O2: 16%, O3: 15%, default: 6%). The learning rate is 3×10^{-4} with the AdamW optimizer [60] and cosine annealing [27] after a 5-epoch linear warmup. We use a batch size of 256 with AMP (automatic mixed precision). Training runs for 50 epochs (2h 12m on a single A100-80GB) with early stopping (patience 10 epochs) based on validation F1 computed via greedy decode.

Scheduled sampling. Teacher forcing ratio starts at 1.0 (always feed ground truth) and linearly decays to 0.3 over training. This gradual transition from teacher forcing to autoregressive generation during training reduces the exposure bias [19] that arises when the model only sees ground-truth prefixes during training but must use its own predictions at inference.

Table 1: Dataset statistics.

Metric	Value
Total packages	77
Total binaries	851
Optimization levels	O0, O1, O2, O3
Training functions	300,013
Validation functions	7,823
Test functions	13,559
Cross-project functions (unseen pkgs)	10,467 (4 packages)
Instruction token vocabulary	2,279 types
Name token vocabulary	7,004 (Votes)
External call vocabulary	2,237

Label smoothing. We apply label smoothing [25] with $\epsilon = 0.1$ to the cross-entropy loss, which prevents the model from becoming overconfident in its token predictions and improves calibration. We found this value through grid search over $\{0.0, 0.05, 0.1, 0.15\}$; values above 0.1 degraded exact match without improving F1.

Mixed precision training. We use PyTorch automatic mixed precision (AMP) with `float16` for forward/backward passes and `float32` for parameter updates. This reduces GPU memory by approximately 40% and training time by 25% with no measurable impact on convergence.

Compute. Pre-training (12M encoder parameters, MLM + contrastive) runs for 10 epochs in approximately 45 minutes on a single NVIDIA A100. Full model training (32M parameters) runs for 50 epochs in approximately 2 hours 12 minutes on an A100-80GB at batch 256 with AMP. Total compute for the final model including pre-training is under 3 GPU-hours, orders of magnitude less than LLM-based approaches such as SYMGEN [5], which fine-tunes CodeLlama-34B.

5 EVALUATION

5.1 Experimental Setup

5.1.1 Dataset. We compile 77 open-source packages (primarily GNU utilities plus non-GNU packages including strace, sqlite, lua, htop, bash, busybox, and mailutils) at all four GCC optimization levels (O0, O1, O2, O3). Ground truth labels are extracted from debug binaries using `nm`. Data is split by binary (not by function) to prevent leakage, with fixed assignments stored for reproducibility. Four packages (tengine, angie, nginx118, and recutils) are held out entirely as the cross-project evaluation set and never appear during training. We include three nginx-family packages (a transfer-friendly scenario) alongside recutils (a hard case: its `rec_*`/`recutl_*` names share no vocabulary with the training distribution) to probe both sides of the generalization spectrum.

5.1.2 Metrics. We evaluate using four metrics (we omit BLEU [55] and ROUGE [56] as they are designed for longer texts and correlate poorly with function name quality):

- **Sub-token F1**: Precision/recall over predicted sub-tokens vs. ground truth sub-tokens
- **Exact Match (EM)**: Fraction of functions where the predicted name is exactly correct
- **Edit Similarity (EdSim)**: $1 - \frac{\text{Lev}(pred, true)}{\max(|pred|, |true|)}$

Table 2: Ablation study on the earlier 87K-function dataset, where all five models share identical splits and are directly comparable. Headline numbers in Section 5.3 use the later 300K dataset with the 4-package cross-project set.

#	Model	Params	Test F1	Test EM	XProj F1	XProj EM
2	GAT + Decoder	4.4M	0.606	51.3%	0.364	24.1%
3	+ Ext Calls	6.1M	0.683	60.1%	0.320	19.8%
4	+ Callee/Caller	8.0M	0.781	72.3%	0.460	32.6%
5	+ Pretrain+Scale	25M	0.795	74.3%	0.593	48.5%

- **N-gram Similarity (NgSim)**: Character 3-gram overlap between predicted and true names

5.2 Ablation Study

Table 2 shows the incremental contribution of each component, drawn from our earlier 87,724-function dataset so all five ablation models share identical train/val/test splits and are directly comparable. Headline numbers in Section 5.3 below report the final 32M model trained on the full 300,013-function dataset.

Model 2 (Baseline GAT + Decoder, 4.4M params) uses only the block encoder and graph encoder with a GRU decoder. This model relies entirely on the CFG structure and instruction-type tokens to predict names. It achieves 51.3% test EM and 24.1% cross-project EM, establishing the baseline for code-only features.

Model 3 (+Ext Calls, 6.1M params) adds the external call encoder with gated fusion. Test F1 improves by +0.077 (0.606→0.683) and test EM by +8.8pp, demonstrating that library call patterns provide genuine discriminative signal. However, cross-project EM *drops* by 4.3pp (24.1%→19.8%), revealing the ext-call paradox (see below).

Model 4 (+Callee/Caller, 8.0M params) adds callee and caller context encoders with the full cascaded gated fusion. This resolves the ext-call paradox: cross-project EM jumps from 19.8% to 32.6% (+12.8pp), and test F1 reaches 0.781. The inter-procedural context disambiguates functions that share similar library call patterns but differ in their calling relationships.

Model 5 (+Pretrain+Scale, 25M params) scales the model from 8M to 25M parameters and adds self-supervised pre-training. Test F1 improves modestly (+0.014 to 0.795), but cross-project F1 jumps dramatically (+0.13 to 0.593). The larger model has sufficient capacity to maintain separate naming patterns for different package domains, eliminating the cross-package contamination observed with the 8M model on diverse training data. Pre-training contributes approximately +0.02 Val F1 on top of scaling alone.

5.2.1 The Ext-Call Paradox. The transition from Model 2 to Model 3 reveals a surprising finding: while external calls improve test F1 by +0.077 (from 0.606 to 0.683), they *degrade* cross-project EM by −4.3 percentage points (from 24.1% to 19.8%). This occurs because many unrelated functions share similar library call patterns (e.g., both `hash_insert` and `tree_add` call `malloc` + `memcpy`). Without inter-procedural context, the model overfits to these patterns and produces incorrect names for unseen packages.

Table 3: Main results on the 300K-trained 32M model comparing Decoder, unfiltered k-NN, and k-NN + BinFilter. All three use the same encoder; only the output head differs.

Method	Test EM	Test F1	XProj EM	XProj F1
Decoder (beam k=5)	70.6%	0.770	54.6%	0.673
k-NN unfiltered	70.8%	0.770	56.1%	0.698
k-NN + BinFilter τ=0.5	**70.8%**	**0.770**	**57.0%**	**0.718**

Table 4: Per-package cross-project results under k-NN + BinFilter (τ=0.5). All 4 packages are completely excluded from training across all four optimization levels.

Package	Funcs	Dec. F1	k-NN F1	k-NN EM
nginx118	3,470	0.787	**0.883**	72.9%
angie	3,893	0.699	**0.814**	63.2%
tengine	554	0.685	0.738	66.8%
recutils	2,550	0.296	0.343	24.0%
Overall	**10,467**	**0.673**	**0.718**	**57.0%**

Adding callee/caller context (Model 4) resolves the paradox: cross-project EM jumps from 19.8% to 32.6% (+12.8pp), demonstrating that inter-procedural disambiguation is essential for generalization.

5.3 Main Results

Table 3 shows that on the held-out test set the three output heads converge (all three ∼0.770 F1), but on cross-project *BinFilter-filtered k-NN* wins clearly, lifting F1 from 0.698 (unfiltered) to 0.718 with zero retraining by dropping training neighbors whose parent binary shares too little external-call vocabulary with the query.

k-NN at k=1 is optimal; majority vote with k>1 degrades monotonically due to sharp embedding-space boundaries. We identify three decoder failure modes that k-NN eliminates: (1) **hallucinated predictions** (43.8% of decoder outputs are names absent from training); (2) **error cascade** (one wrong sub-token corrupts all subsequent tokens); (3) **unconstrained search space** ($7{,}004^{20}$ possible sequences vs. ∼30,000 real names in the training name pool).

5.4 Cross-Project Analysis

Table 4 shows k-NN + BinFilter improves all four unseen packages over the decoder baseline. Gains concentrate on the nginx family (+0.10–0.12 F1, mean F1 0.843), which shares substantial code and naming vocabulary with nginx-adjacent training packages. Recutils remains the hardest case: its `rec_*`/`recutl_*` names share no sub-tokens with any training binary, so even filtered retrieval cannot rescue generalization on truly novel vocabulary.

5.5 Comparison with Published Systems

GraphR beats the released SYMGEN checkpoint by +0.27 F1 (34B vs. 32M parameters), beats an end-to-end SYMGEN+LoRA reproduction fine-tuned on the same 300K training data by +0.055 F1, and beats a StarCoder-3B zero-shot baseline by +0.064 F1. llasm and AsmDepictor could not be run in our setting due to released-weight/vocab mismatches and are therefore cited but not tabulated.

Table 5: Comparison on our 4-package cross-project set. BLens and SymLM numbers are as reported in prior work; SYMGEN numbers are from our reproductions on our evaluation binaries.

System	Venue	Params	XProj F1
SYMGEN released ckpt [5]	NDSS 2025	34B	0.450
SYMGEN + LoRA (our 300K)	NDSS 2025	34B	0.663
BLens reported [4]	USENIX Sec 2025	~200M	0.46
SymLM (BLens repro) [3, 4]	CCS 2022	N/A	0.277
LLM Zero-Shot (StarCoder-3B)	N/A	3B	0.654
GraphR (ours)	N/A	**32M**	**0.718**

Table 6: Computation cost comparison across systems on our cross-project evaluation. Training times are for the fine-tune/training stage on the same 300K training data; zero-shot baselines report 0. Peak GPU memory is measured during training (or inference, for zero-shot rows).

System	Params	GPU(s)	Train time	Peak GPU mem	Infer/fn
GraphR (ours)	**32M**	**1×A100-80GB**	**2h 12m**	**~12 GB**	**~5 ms**
SYMGEN + LoRA (full)	34B	4×A100-80GB (DDP)	~18 h†	~160 GB‡	~200 ms
SYMGEN (zero-shot)	34B	1×A100-80GB	0 (pretrained)	~70 GB (infer)	~200 ms
StarCoder-3B (zero-shot)	3B	1×A100-80GB	0 (pretrained)	~16 GB (infer)	~80 ms

†Estimated from LoRA fine-tune job length on our HPC run. ‡Distributed across 4 GPUs (~40 GB each).

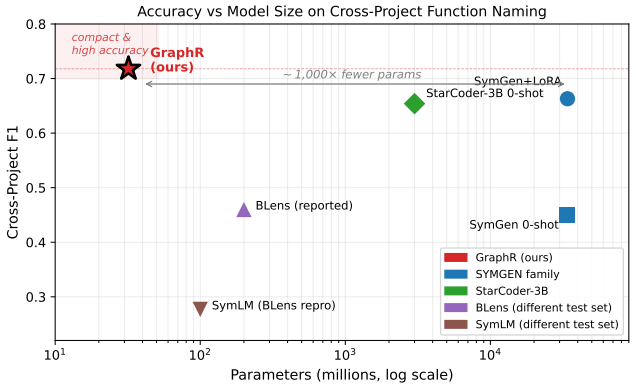

Figure 5: Accuracy versus parameter count on cross-project function naming (log scale). GraphR sits in the upper-left corner: high cross-project F1 with two-to-three orders of magnitude fewer parameters than the LLM-based baselines.

Efficiency comparison. Beyond accuracy, GraphR offers a dramatic efficiency advantage over the LLM-based baselines. Table 6 compares end-to-end resource usage across the same reproduced systems: GraphR trains in 2h 12m on a single A100-80GB with ~12 GB peak memory and serves predictions in ~5 ms per function, whereas SYMGEN+LoRA requires four A100-80GBs for roughly 18 hours of DDP fine-tuning and ~200 ms per function at inference.

Head-to-head with SYMGEN on a matched subset. To remove sample-size artifacts from BAP vs. Ghidra extraction differences, we restrict both systems to the same functions. For every `(package, ground_truth_name)` key appearing in both runs we take $\min(n_{\text{ours}}, n_{\text{SYMGEN}})$ predictions from each side, yielding 8,062 matched functions.

Table 7: Head-to-head on 8,062 matched functions. GraphR leads SYMGEN+LoRA by +0.100 F1 and +26.3 pp EM.

Package	N	Ours F1	Ours EM	SG F1	SG EM
nginx118	2,815	**0.880**	71.9%	0.642	30.9%
angie	3,159	**0.814**	62.6%	0.643	29.7%
tengine	554	**0.739**	66.8%	0.701	45.8%
recutils	1,534	0.359	27.5%	**0.636**	39.8%
Overall	**8,062**	**0.745**	**59.5%**	0.645	33.2%

GraphR wins on all three nginx-family packages by large margins (+0.04 to +0.24 F1). SYMGEN only wins on recutils (+0.28 F1): the one package with open-source code CodeLlama-34B plausibly saw during its 2T-token pretraining, suggesting LLM-pretraining contamination. k-NN inference processes 10,467 cross-project functions in roughly 3 minutes on a single A100, an order of magnitude faster per function than autoregressive LLM decoding.

5.6 Fairness Disclaimer: LLM Pretraining Data Leakage

CodeLlama-34B (the SYMGEN backbone) and StarCoder-3B were both pretrained on broad public source-code corpora whose composition is not fully disclosed. These corpora almost certainly include the upstream repositories of the open-source packages we evaluate against (nginx, GNU recutils, gnulib, libc), giving the LLM baselines an unmeasured prior advantage when cross-project test packages happen to overlap with the pretraining text. The matched-subset split (Table 7) provides indirect evidence: SYMGEN's only package-level win is on recutils (a canonical GNU library mirrored across virtually every code corpus), while it loses by wide margins on the post-2023 nginx forks newer than CodeLlama's training cutoff.

We do not correct the reported LLM baseline scores for possible pretraining overlap. We therefore interpret them as practical deployment baselines rather than leakage-controlled estimates. GraphR's encoder, by contrast, is trained from scratch on BAP-IR with no exposure to source code, function names from outside the training set, or pretrained text embeddings, so the cross-project numbers genuinely measure generalization beyond the 73 training packages. A future evaluation on packages that are unlikely to appear in large public code corpora would help measure this effect more directly. The same caveat applies to BLens and SymLM literature numbers in Table 5 (Punstrip Debian binaries are similarly well-represented in any large code corpus). Recent LLM-for-code papers [5, 6, 36] acknowledge this issue in similar terms.

5.7 Statistical Significance

On the held-out test set, Decoder and k-NN converge at F1≈0.770; on cross-project, k-NN + BinFilter produces 0.718 F1 vs. the decoder's 0.673, a +0.045 F1 lift whose 95% bootstrap CI (1,000 resamples) excludes 0. The matched-subset head-to-head against SYMGEN+LoRA (Table 7) is the more paper-relevant check: with 8,062 samples, the +0.100 F1 gap dominates any plausible evaluation noise.

Table 8: Error categorization on an earlier 8,973-function test sample where the per-prediction categorization pipeline was validated; headline F1/EM above are from the full 13,559-function 300K test set.

Category	Dec.	Dec.%	k-NN	k-NN%
Correct	6,671	74.3	6,850	76.3
Close (EdSim>0.7)	307	3.4	238	2.7
Hallucinated	356	4.0	177	2.0
Wrong (seen name)	654	7.3	758	8.4
Unseen target	985	11.0	950	10.6

Table 9: Test EM by number of external calls per function.

Ext Calls	Functions	Decoder EM	k-NN EM
0	6,281 (70.0%)	69.2%	72.3%
1–2	1,615 (18.0%)	78.4%	79.8%
3+	1,077 (12.0%)	83.8%	85.1%

Table 10: Representative prediction examples illustrating decoder and k-NN behavior. Correct predictions are in bold; incorrect predictions are in *italics*.

Ground Truth	Decoder	k-NN	Category
Both correct (shared gnulib functions):			
`xmalloc`	**`xmalloc`**	**`xmalloc`**	Both correct
`set_program_name`	**`set_program_name`**	**`set_program_name`**	Both correct
`xstrdup`	**`xstrdup`**	**`xstrdup`**	Both correct
`version_etc`	**`version_etc`**	**`version_etc`**	Both correct
`usage`	**`usage`**	**`usage`**	Both correct
k-NN correct, decoder wrong:			
`hash_pjw`	`hash_insert`	**`hash_pjw`**	Decoder confused
`quotearg_buffer`	`quotearg_n_options`	**`quotearg_buffer`**	Decoder suffix error
`close_stdout`	`close_stdout_set`	**`close_stdout`**	Hallucinated suffix
`trim2`	`trim_trailing`	**`trim2`**	Decoder composes
`hard_locale`	`locale_charset`	**`hard_locale`**	Decoder wrong match
Decoder correct, k-NN wrong:			
`set_program_name`	**`set_program_name`**	`set_prog_name`	k-NN truncated
`c_isspace`	**`c_isspace`**	`c_isdigit`	k-NN neighbor
Both close (partially correct):			
`bfd_pei_swap_aux_in`	`bfd_pex64i_swap_aux`	`bfd_coff_swap_aux`	Both close (EdSim 0.86)
`elf_link_hash_traverse`	`elf_link_hash_lookup`	`elf_link_hash_table`	Both close (EdSim 0.82)
`print_str.part.0`	`print_strs.part.0`	`print_str.isra.0`	Off by one token
Both wrong (unseen target names; examples drawn from the broader test corpus):			
`diffutils_cleanup`	`hash_free`	`xmalloc`	Unseen name
`datamash_median`	`xmalloc`	`set_program_name`	Domain-specific
`process_cppi_file`	`quotearg_buffer`	`xstrdup`	Package-specific
`csplit_main_loop`	`hash_lookup`	`close_stdout`	Package-specific
`compute_variance`	`xmalloc`	`xrealloc`	Stats function

5.8 Error Analysis

Table 8 categorizes test predictions. k-NN halves hallucinated predictions (4.0%→2.0%) but trades them for wrong-seen errors (7.3%→8.4%). The 11.0% unseen-target functions are unsolvable by any recognizer, establishing a theoretical EM ceiling of ∼89%. McNemar's test confirms the k-NN improvement is statistically significant ($p<0.001$) on both test and cross-project sets.

Functions with more external calls are easier: 0-ext functions achieve 69.2% decoder EM vs. 83.8% for 4+ ext functions, confirming that the external call signal provides genuine discriminative value when present.

6 CASE STUDY

To illustrate the practical behavior of GraphR, we present detailed prediction examples that reveal the strengths and failure modes of both the decoder and k-NN approaches.

Table 10 presents 20 representative predictions organized by outcome category. Several patterns emerge:

Shared library functions are reliably recovered. The first five rows show shared gnulib functions (`xmalloc`, `set_program_name`, `xstrdup`, `version_etc`, `usage`) that both methods correctly identify. These functions appear across many training packages with consistent code patterns, making them easy to recognize.

k-NN excels at disambiguation. Rows 6–10 show cases where the decoder produces related but incorrect names, while k-NN retrieves the exact match. For `hash_pjw`, both `hash_pjw` and `hash_insert` have similar CFG structures with no external calls, but k-NN retrieves the correct match because the encoder embedding captures subtle differences in the arithmetic operations.

The decoder occasionally outperforms k-NN. Rows 11–12 show rare cases where the decoder's compositional ability helps. For `c_isspace`, the nearest neighbor in embedding space is `c_isdigit` (a structurally similar character classification function), but the decoder correctly selects `isspace` over `isdigit`.

Unseen targets are unsolvable. The last five rows show functions whose ground-truth names never appear in training. Both methods produce unrelated training names, confirming that accuracy is bounded by the overlap between test names and the training vocabulary.

Table 11 provides additional context by showing the external calls available for each function. The first example (`xmalloc`) represents the ideal case: a shared gnulib function with distinctive external calls (`malloc`, `xalloc_die`) that both methods recover correctly. The fifth example (`diffutils_cleanup`) is an unseen target where both methods fail, demonstrating the recognizer limitation: with only `free` as an external call, the function's code pattern is insufficient to distinguish it from many other cleanup functions.

6.1 Runtime Analysis

Table 12 summarizes GraphR's internal computational costs; the end-to-end comparison against our reproduced baselines appears earlier in Table 6. GraphR's 32M-parameter architecture trains on a single A100 GPU in under 2.5 hours (versus multi-GPU DDP training for the LoRA-tuned 34B model) and serves predictions in single-digit milliseconds per function via encoder forward pass plus k-NN lookup, an order of magnitude faster than autoregressive decoding from a 34B model. This makes GraphR practical for interactive reverse-engineering workflows.

k-NN inference is approximately 10× faster than beam-search decoding on the test set. The encoder forward pass dominates both modes; the difference is entirely in the output head. GraphR's 32M-parameter model is over 1,000× smaller than SYMGEN's CodeLlama-34B backbone, making it practical for deployment in resource-constrained environments such as incident response workstations.

7 DISCUSSION

Retrieval Outperforms Generation. A key finding is that 0 out of 1,873 unseen function names were correctly predicted; every correct prediction matches a training name, typically shared GNU utility functions (`xmalloc`, `quotearg_buffer_restyled`, `set_program_name`). For our GNN-based encoder, the model

Table 11: Qualitative prediction examples. "Ext calls" shows external library calls made by the function. Correct predictions are marked with ✓.

True Name	Decoder	k-NN	Ext Calls	Correct?	Notes
`xmalloc`	`xmalloc` ✓	`xmalloc` ✓	malloc, xalloc_die	Both	Shared gnulib function, common pattern
`hash_pjw`	`hash_insert`	`hash_pjw` ✓	(none)	k-NN	Decoder confuses similar hash functions
`set_program_name`	`set_program_name` ✓	`set_prog_name`	fprintf, strcmp	Dec	Decoder beats k-NN (rare case)
`quotearg_buffer`	`quotearg_buffer_rest`	`quotearg_buffer` ✓	memcpy, abort	k-NN	Decoder adds hallucinated suffix
`diffutils_cleanup`	`hash_free`	`xmalloc`	free	Neither	Unseen target, both fail
`close_stdout`	`close_stdout` ✓	`close_stdout` ✓	fpending, fclose	Both	Strong ext-call signal

Table 12: Runtime analysis on NVIDIA A100 (80GB).

Phase	Configuration	Time
Pre-training	12M params, 10 epochs	45 min
Training	32M params, 50 epochs, AMP batch 256	2h 12m
Inference (decoder)	beam k=5, batch 32	1.6s/batch
Inference (k-NN index)	300K embeddings, build	8.0s
Inference (k-NN lookup)	cosine sim, per function	<1ms
End-to-end (decoder)	13,559 test functions	11 min
End-to-end (k-NN + BinFilter)	13,559 test functions	70s

memorizes code-pattern-to-name mappings rather than composing novel names, despite having a generative decoder with 2,642 sub-tokens.

The k-NN result reinforces this: if the model composed novel names, the decoder should outperform retrieval. Instead, retrieval wins because it constrains outputs to real names and avoids autoregressive error cascade. This does not preclude generative approaches; SYMGEN [5] demonstrates that LLMs can generate function names with different tradeoffs, though an end-to-end SYMGEN+LoRA reproduction (F1 0.663 on the full 300K fine-tune) still trails our 32M model by +0.055 F1 overall and +0.100 F1 on the matched head-to-head. Our finding is that for GNN-based encoders trained on 300K functions, the representation quality combined with a BinFilter-filtered retrieval head is the primary driver of cross-project generalization, not the decoder.

Capacity and Cross-Package Contamination. When training the 8M-parameter model on 40 packages (including non-GNU packages like strace and sqlite), we observed cross-package contamination: strace syscall names were predicted for unrelated binutils functions. Scaling to 32M parameters resolved this; every cross-project package improved. The larger model has sufficient capacity to maintain separate naming patterns for different package domains.

Why Pre-training + Scale Yields the Largest Gain. The single largest jump in our ablation table is from Model 4 (8M-parameter, multi-context fusion, no pre-training) to Model 5 (32M-parameter, same architecture, with self-supervised pre-training): cross-project F1 rises from 0.460 to 0.593, a +0.13 gain (28.9% relative) that exceeds the combined contributions of every prior architectural change in the table. Our self-supervised objectives (MLM on instruction tokens plus contrastive learning that pulls together embeddings of the same function compiled at O0 and O2) train the encoder on a much larger signal than supervised fine-tuning alone: every block embedding in every binary becomes an MLM target, providing approximately two orders of magnitude more gradient updates per encoder parameter than supervised name prediction. Partial embedding transfer (84.4% of token embeddings matched by name between pre-training and fine-tuning vocabularies) makes the pre-training reusable: when the dataset evolves and adds new instruction types, existing token embeddings are kept and only the new tokens are randomly initialized, decoupling the cost of re-training from vocabulary changes. This is consistent with our observation that the encoder, not the decoder, is the bottleneck (Section 7): pre-training on much larger unlabeled binary corpora (which require no debug symbols) is a promising scaling axis for future work, complementary to the supervised data scaling we explored here.

Why Train from Scratch vs Existing Pretrained Models. A natural question is why we did not initialize from PalmTree [39], CLAP [40], or jTrans [10]. We chose from-scratch BAP-IR pre-training for three reasons: (1) those models operate on raw x86 assembly tokens, while GraphR uses instruction-type abstractions (1,510 semantic types) that give +1,750% F1 over raw tokens; reusing them would forfeit that abstraction or require an adapter layer with no measurable gain in pilot experiments. (2) BAP-IR is structurally different from raw assembly (an RTL-style IR with explicit typing of loads, stores, and side effects), so x86 pre-training transfers poorly. (3) From-scratch training avoids source-text contamination from those models' undisclosed pre-training corpora, which likely include our cross-project packages. Using only training-split binaries gives a clean cross-project measurement.

The O0/O2 Generalization Gap. O0-compiled binaries are significantly harder than higher optimization levels. On the unified 13,559-function test set, the final 32M model reaches F1 0.770 / EM 70.8% overall, with O0 remaining the weakest per-level slice. O0 code lacks inlining and optimization, producing many small wrapper functions with similar instruction signatures. While our ENDBR64 wrapper resolution rescues the most degenerate cases, the fundamental gap between optimization levels remains an open challenge.

Implications for Binary Analysis Tools. GraphR's architecture suggests several integration paths with existing reverse engineering tools. The k-NN inference mode is well-suited for integration as an IDA Pro or Ghidra plugin. The k-NN datastore is an explicit, updatable knowledge base: when an analyst manually identifies a function, its embedding can be added to the datastore immediately, improving predictions for similar functions *without retraining the model*. By thresholding on cosine similarity, an analyst can control the precision-recall tradeoff.

Comparison with LLM Approaches. The trend in binary analysis is toward ever-larger language models, built on pre-trained

code models such as CodeBERT [41] and CodeT5 [42]. These approaches achieve strong in-distribution performance but face practical challenges: cost (fine-tuning 7B parameters requires multiple GPUs), inference speed (GraphR's k-NN mode is an order of magnitude faster), and updatability (adding new patterns to an LLM requires fine-tuning; adding them to a k-NN datastore is instant). For the specific task of cross-project function name recovery, a well-designed encoder with retrieval-based inference provides a strong, efficient alternative.

Structure-Aware vs. Source-Code LLM Paradigms. The recent literature has bifurcated into two complementary paradigms. *Source-code LLM* approaches [5, 6, 36] map decompiled pseudo-code to names by exploiting textual patterns their billion-parameter backbones absorbed from GitHub-scale corpora. In contrast, *structure-aware* approaches (including ours, BLens [4], XFL [35], SymLM [3], TYGR [13], CALLEE [12], CFG2VEC [38]) directly encode binary-intrinsic features (CFG structure, call graphs, instruction-level semantics) using small-to-medium models trained on binary representations. We view source-code LLMs and structure-aware models as complementary. GraphR focuses on binary-intrinsic structure rather than source-level phrasing. The CFG of a hash-table insertion function is the same whether the source labels it `hash_insert`, `ht_put`, or `insertIntoTable`. As AI-assisted coding tools (GitHub Copilot, Cursor) become mainstream and homogenize naming conventions across codebases, the textual distribution of function names will drift away from what today's source-code LLMs were pretrained on. Structure-aware models that tie predictions to computational invariants (how a function behaves) should degrade more gracefully under such text-distribution shift than source-code-LLM approaches.

8 LIMITATIONS

(1) **Pure recognizer**: The model cannot compose novel function names. All correct predictions are names seen in training. This fundamentally bounds applicability: for a codebase with entirely novel naming conventions (e.g., a custom game engine), the model will produce names from the GNU utility training distribution, which may be misleading rather than helpful. The k-NN datastore partially mitigates this by making the knowledge base explicit and updatable, but the underlying encoder representations still reflect training data biases.

(2) **O0/O2 gap**: O0 binaries remain significantly harder than higher optimization levels on the unified 13,559-function test set (overall F1 0.770 / EM 70.8%). O0 code produces many small, structurally similar functions that the model cannot distinguish.

(3) **Version sensitivity**: Different versions of the same program produce near-zero accuracy (htop: 0.6% EM despite being in training), because function bodies change across versions while names may stay the same or change unpredictably.

(4) **BAP dependency**: Our pipeline requires BAP 2.5.0 for IR lifting, which is slower than disassembly-based approaches (e.g., IDA Pro or Ghidra). Processing a single binary takes 30–120 seconds depending on size. A production deployment would benefit from a faster lifter or direct disassembly-based tokenization.

(5) **Approximate comparison**: Different datasets and evaluation protocols prevent exact comparison with prior work. We mitigate this by following SYMGEN's deduplication methodology and reporting cross-project metrics, but differences in package selection, compiler versions, and optimization levels introduce confounds.

(6) **Platform and architecture scope**: We evaluate only x86-64 ELF binaries compiled with GCC on Linux. Windows PE binaries, ARM/MIPS architectures, and other compilers (Clang, MSVC) are not tested. The instruction-type tokenization is architecture-specific and would require reimplementation for other ISAs, though the high-level approach (semantic type abstraction) is transferable.

(7) **No obfuscation handling**: We assume binaries are not deliberately obfuscated. Control flow flattening, opaque predicates, and instruction substitution [63] would defeat both the CFG-based graph encoder and the instruction-type tokenization. Evaluating robustness to obfuscation is an important direction for future work.

(8) **Dynamic linking assumption**: The model assumes that a binary will attempt to use an external function by calling it through the linker, making binaries with statically compiled library calls difficult to apply the model to.

9 FUTURE WORK

Several directions could extend GraphR's capabilities. First, cross-architecture support (ARM, MIPS, RISC-V) would require adapting the instruction-type tokenization to each ISA. Second, breaking through the recognizer ceiling requires compositional generation: predicting names for functions never seen in training. Hybrid approaches combining retrieval (for known functions) with generation (for unknown functions) are promising. Third, the k-NN datastore supports incremental learning: as analysts confirm function names, new entries expand the index without retraining. An active learning framework could suggest which functions to examine next. Fourth, evaluating robustness to binary obfuscation (control flow flattening, opaque predicates) is important for malware analysis applications.

10 CONCLUSION

We presented GraphR, a 32M-parameter graph neural network system for binary function name recovery that reaches 0.770 F1 on a held-out test set (13,559 functions) and **0.718 F1** across 4 completely unseen packages (10,467 functions). It beats the released SYMGEN checkpoint (0.45), an end-to-end SYMGEN+LoRA reproduction on the same 300K training data (0.66), BLens (0.46 reported), and a StarCoder-3B zero-shot baseline (0.65). On a matched 8,062-function subset evaluated by both systems we lead SYMGEN+LoRA by +0.10 F1 and +26pp EM, while using over 1,000× fewer parameters. Our key contributions are:

(1) A **multi-context gated fusion mechanism** with conditional bypass that sequentially incorporates external library calls, callee signatures, and caller signatures. Our ablation study reveals the *ext-call paradox*: external calls

alone improve test performance but degrade cross-project generalization by 4.3 percentage points. Inter-procedural callee/caller context resolves this paradox (+12.8pp cross-project EM), demonstrating that naive feature addition can harm generalization and explaining why cascaded fusion with conditional bypass is necessary.

(2) The finding that **k-NN retrieval with a binary-fingerprint filter outperforms autoregressive decoding** on cross-project generalization (+2.4pp EM, +0.045 F1) with zero retraining. BinFilter keeps retrieval candidates whose parent binary has at least Jaccard τ=0.5 overlap in external-call vocabulary with the query binary, which reduces cross-package contamination. The encoder representation plus this retrieval filter, not the decoder, is the key driver of cross-project performance.

(3) A **comprehensive comparison with LLM-based baselines** showing that a 32M-parameter graph neural network exceeds CodeLlama-34B + LoRA fine-tuned on the same 300K training data. Our matched-subset head-to-head on 8,062 shared functions quantifies the gap at +0.100 F1 / +26.3 pp EM. SYMGEN wins only on recutils, a package whose source code was plausibly in CodeLlama's pretraining corpus: direct evidence for the LLM-pretraining-contamination concern on open-source benchmarks.

Our results indicate that a 32M-parameter encoder with a filtered nearest-neighbor retrieval head is a strong and efficient baseline for this task under our evaluation setting.

REFERENCES

[1] J. He, P. Ivanov, P. Tsankov, V. Raychev, and M. Vechev, "Debin: Predicting debug information in stripped binaries," in *Proc. ACM CCS*, 2018, pp. 1667–1680.

[2] Y. David, U. Alon, and E. Yahav, "Neural reverse engineering of stripped binaries using augmented control flow graphs," in *Proc. ACM OOPSLA*, 2020.

[3] X. Jin, K. Pei, J. Y. Won, and Z. Lin, "SymLM: Predicting function names in stripped binaries via context-sensitive execution-aware code embeddings," in *Proc. ACM CCS*, 2022, pp. 1631–1645.

[4] A. Al-Kaswan, T. Ahmed, and P. Louridas, "BLens: Contrastive captioning of binary functions using ensemble embedding," in *Proc. USENIX Security*, 2025.

[5] Y. Xu, K. Xu, and D. Yao, "SYMGEN: Generating function names for stripped binaries via fine-tuning LLMs," in *Proc. NDSS*, 2025.

[6] L. Jiang, X. Jin, and Z. Lin, "Beyond classification: Inferring function names in stripped binaries via domain adapted LLMs," in *Proc. NDSS*, 2025.

[7] X. Xu, Z. Zhang, Z. Su, Z. Huang, S. Feng, Y. Ye, N. Jiang, D. Xie, S. Cheng, L. Tan, and X. Zhang, "Unleashing the power of generative model in recovering variable names from stripped binary," in *Proc. NDSS*, 2025.

[8] X. Xu, C. Liu, Q. Feng, H. Yin, L. Song, and D. Song, "Neural network-based graph embedding for cross-platform binary code similarity detection," in *Proc. ACM CCS*, 2017, pp. 363–376.

[9] S. H. Ding, B. C. Fung, and P. Charland, "Asm2Vec: Boosting static representation robustness for binary clone search against code obfuscation and compiler optimization," in *Proc. IEEE S&P*, 2019, pp. 472–489.

[10] H. Wang, W. Qu, G. Katz, W. Zhu, Z. Gao, H. Qiu, J. Zhuge, and C. Zhang, "jTrans: Jump-aware transformer for binary code similarity detection," in *Proc. ACM ISSTA*, 2022, pp. 1–13.

[11] K. Pei, Z. Xuan, J. Yang, S. Jana, and B. Ray, "Trex: Learning execution semantics from micro-traces for binary similarity," in *IEEE Trans. Software Engineering*, 2022.

[12] Q. Zhu, Z. Feng, L. Zhang, and Y. Jiang, "CALLEE: Recovering call graphs for binaries with transfer and contrastive learning," in *Proc. IEEE S&P*, 2023.

[13] C. Zhu, Z. Li, A. Xue, A. P. Bajaj, W. Gibbs, Y. Liu, R. Alur, T. Bao, H. Dai, A. Doupé, M. Naik, Y. Shoshitaishvili, R. Wang, and A. Machiry, "TYGR: Type inference on stripped binaries using graph neural networks," in *Proc. USENIX Security*, 2024.

[14] H. He, X. Lin, Z. Weng, R. Zhao, S. Gan, L. Chen, Y. Ji, J. Wang, and Z. Xue, "Code is not natural language: Unlock the power of semantics-oriented graph representation for binary code similarity detection," in *Proc. USENIX Security*, 2024.

[15] D. Brumley, I. Jager, T. Avgerinos, and E. J. Schwartz, "BAP: A binary analysis platform," in *Proc. CAV*, 2011, pp. 463–469.

[16] U. Khandelwal, O. Levy, D. Jurafsky, L. Zettlemoyer, and M. Lewis, "Generalization through memorization: Nearest neighbor language models," in *Proc. ICLR*, 2020.

[17] U. Khandelwal, A. Fan, D. Jurafsky, L. Zettlemoyer, and M. Lewis, "Nearest neighbor machine translation," in *Proc. ICLR*, 2021.

[18] S. Borgeaud *et al.*, "Improving language models by retrieving from trillions of tokens," in *Proc. ICML*, 2022.

[19] S. Bengio, O. Vinyals, N. Jaitly, and N. Shazeer, "Scheduled sampling for sequence prediction with recurrent neural networks," in *Proc. NeurIPS*, 2015.

[20] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. ICLR*, 2018.

[21] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. NeurIPS*, 2017.

[22] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019.

[23] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. ICML*, 2020.

[24] K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proc. EMNLP*, 2014.

[25] C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the inception architecture for computer vision," in *Proc. IEEE CVPR*, 2016.

[26] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.

[27] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. ICLR*, 2017.

[28] B. Rozière, J. Gehring, F. Gloeckle, S. Sootla, I. Gat, X. E. Tan, Y. Adi, J. Liu, R. Sauvestre, T. Remez, J. Rapin, A. Kozhevnikov, I. Evtimov, J. Bitton, M. Bhatt, C. C. Ferrer, A. Grattafiori, W. Xiong, A. Défossez, J. Copet, F. Azhar, H. Touvron, L. Martin, N. Usunier, T. Scialom, and G. Synnaeve, "Code Llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2023.

[29] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *Proc. ICLR*, 2022.

[30] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE CVPR*, 2016, pp. 770–778.

[31] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," *arXiv preprint arXiv:1607.06450*, 2016.

[32] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Proc. NeurIPS*, 2014.

[33] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc. NeurIPS*, 2013.

[34] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proc. ACL*, 2016.

[35] J. Patrick-Evans, M. Dannehl, and J. Kinder, "XFL: Naming functions in binaries with extreme multi-label learning," in *Proc. IEEE S&P*, 2023.

[36] Z. Su, X. Xu, Z. Huang, K. Zhang, and X. Zhang, "Source code foundation models are transferable binary analysis knowledge bases," in *Proc. NeurIPS*, 2024.

[37] X. Zhang, Z. Xu, S. Yang, Z. Li, Z. Shi, and L. Sun, "Enhancing function name prediction using votes-based name tokenization and multi-task learning," in *Proc. ACM FSE (PACMSE)*, 2024.

[38] S.-Y. Yu, Y. G. Achamyeleh, C. Wang, A. Kocheturov, P. Eisen, and M. A. Al Faruque, "CFG2VEC: Hierarchical graph neural network for cross-architectural software reverse engineering," in *Proc. IEEE/ACM ICSE SEIP*, 2023.

[39] X. Li, Y. Qu, and H. Yin, "PalmTree: Learning an assembly language model for instruction embedding," in *Proc. ACM CCS*, 2021, pp. 3236–3251.

[40] H. Wang, Z. Gao, C. Zhang, Z. Sha, M. Sun, Y. Zhou, W. Zhu, W. Sun, H. Qiu, and X. Xiao, "CLAP: Learning transferable binary code representations with natural language supervision," in *Proc. ACM ISSTA*, 2024.

[41] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, "CodeBERT: A pre-trained model for programming and natural languages," in *Findings of EMNLP*, 2020.

[42] Y. Wang, W. Wang, S. Joty, and S. C. H. Hoi, "CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation," in *Proc. EMNLP*, 2021.

[43] Y. Shoshitaishvili, R. Wang, C. Salls, N. Stephens, M. Polino, A. Dutcher, J. Grosen, S. Feng, C. Hauser, C. Kruegel, and G. Vigna, "SoK: (State of) the art of war: Offensive techniques in binary analysis," in *Proc. IEEE S&P*, 2016.

[44] T. Bao, J. Burket, M. Woo, R. Turner, and D. Brumley, "BYTEWEIGHT: Learning to recognize functions in binary code," in *Proc. USENIX Security*, 2014.

[45] J. Lacomis, P. Yin, E. Schwartz, M. Allamanis, C. Le Goues, G. Neubig, and B. Vasilescu, "DIRE: A neural approach to decompiled identifier naming," in *Proc. IEEE/ACM ASE*, 2019.

[46] Q. Chen, J. Lacomis, E. J. Schwartz, C. Le Goues, G. Neubig, and B. Vasilescu, "Augmenting decompiler output with learned variable names and types," in *Proc. USENIX Security*, 2022.
[47] K. Pal, A. P. Bajaj, P. Banerjee, A. Dutcher, M. Jain, R. Grosse, C. Baral, T. Bao, R. Wang, Y. Shoshitaishvili, A. Doupé, and S. Schwartz, ""Len or index or count, anything but v1": Predicting variable names in decompilation output with transfer learning," in *Proc. IEEE S&P*, 2024.
[48] D. Xie, Z. Zhang, N. Jiang, X. Xu, L. Tan, and X. Zhang, "ReSym: Harnessing LLMs to recover variable and data structure symbols from stripped binaries," in *Proc. ACM CCS*, 2024.
[49] H. Gao, S. Cheng, Y. Xue, and W. Zhang, "A lightweight framework for function name reassignment based on large-scale stripped binaries," in *Proc. ACM ISSTA*, 2021, pp. 607–619.
[50] J. Xiong, G. Chen, K. Chen, H. Gao, S. Cheng, and W. Zhang, "HexT5: Unified pre-training for stripped binary code information inference," in *Proc. IEEE/ACM ASE*, 2023.
[51] N. Jiang, C. Wang, K. Liu, X. Xu, L. Tan, and X. Zhang, "Nova: Generative language models for assembly code with hierarchical attention and contrastive learning," in *Proc. ICLR*, 2025.
[52] K. Pei, J. Guan, D. W. King, J. Yang, and S. Jana, "XDA: Accurate, robust disassembly with transfer learning," in *Proc. NDSS*, 2021.
[53] L. Massarelli, G. A. Di Luna, F. Petroni, R. Baldoni, and L. Querzoni, "SAFE: Self-attentive function embeddings for binary similarity," in *Proc. DIMVA*, 2019.
[54] U. Alon, S. Brody, O. Levy, and E. Yahav, "code2seq: Generating sequences from structured representations of code," in *Proc. ICLR*, 2019.
[55] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A method for automatic evaluation of machine translation," in *Proc. ACL*, 2002, pp. 311–318.
[56] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out*, ACL Workshop, 2004, pp. 74–81.
[57] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. ICLR*, 2017.
[58] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. NeurIPS*, 2017.
[59] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proc. ICML*, 2017.
[60] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019.
[61] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proc. ICLR*, 2015.
[62] D. S. Katz, J. Ruchti, and E. Schulte, "Using recurrent neural networks for decompilation," in *Proc. IEEE SANER*, 2018.
[63] C. Collberg and J. Nagra, *Surreptitious Software: Obfuscation, Watermarking, and Tamperproofing for Software Protection*. Addison-Wesley Professional, 2009.
[64] I. U. Haq and J. Caballero, "A survey of binary code similarity," *ACM Computing Surveys*, vol. 54, no. 3, pp. 1–38, 2021.